WEEK 2 · DAY 4 · OVERVIEW

LightGBM and Stacking



Created by **Jeff Lenamon**

ABOUT 50 MIN WITH EXERCISES / 12 SECTIONS / 4 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem Statement

BUSINESS CONTEXT

Day 8 gave the desk XGBoost; LightGBM is another widely used boosting library, the same boosting with trees grown another way.

Day 3's forest, Day 8's braked XGBoost and LightGBM now score within a few thousandths of each other on Course 4's card accounts.

A reviewer asks whether one model built on top of them ranks the accounts better than any of them, and how to build it without fooling ourselves.

The accounts are real, from Taiwan in 2005, so they describe consumer credit there and then, not US credit today.

The four person columns are never features, because a model that used them would sort people by them.

OBJECTIVES

Fit LightGBM and score it on training and validation rows, read its trees from a table of nodes and draw one, and count the split points its bins allow.

Measure three of its settings on five folds, stop its trees on rows carved from the training rows, and read its two built-in importances.

Build out-of-fold columns, fit a stacking layer on them, check **`StackingClassifier`** against the hand version, and compare a stack with its members and with plain averages.

ABOUT 50 MIN WITH EXERCISES / 12 SECTIONS / 4 EXERCISES

Your Route Through Day 4

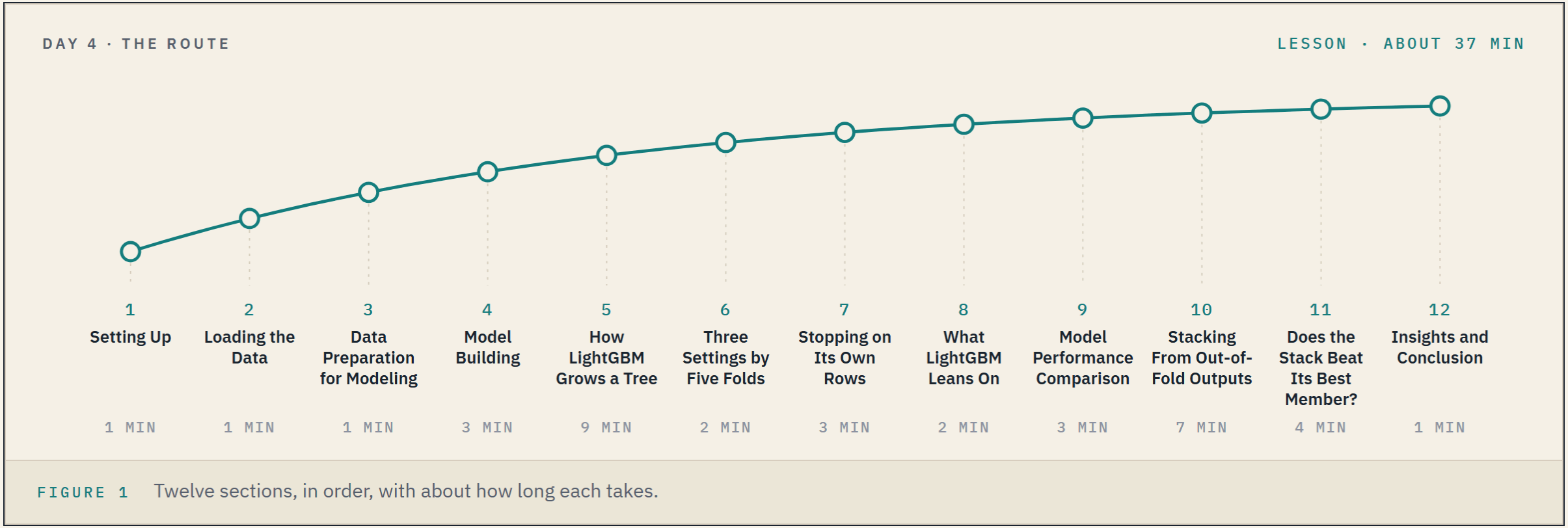


FIGURE 1 Twelve sections, in order, with about how long each takes.

Time. About 37 minutes for the lesson, then about 12 for the four exercises.

Fix the seed and load the libraries.

Setting Up

THE SETUP

```
import os
from pathlib import Path
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["OPENBLAS_NUM_THREADS"] = "1"
SEED = 42
DATA = "../../../data/"
if not Path(DATA + "benchmark.csv").exists():
```

TODAY'S LIBRARY BESIDE THE EARLIER MODELS

```
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from xgboost import XGBClassifier
import lightgbm as lgb
from lightgbm import LGBMClassifier
```

Run this cell and move on: it fixes the seed and finds the course data.

Read the card file and pick 19 columns.

Loading the Data

THE FILE'S COLUMNS

- **LIMIT_BAL**: credit limit, NT dollars
- **PAY_0**, **PAY_2** to **PAY_6**: repayment status, September back to April 2005, where -1 is paid on time, 1 to 9 is months late, and -2 and 0 are undocumented (used as numbers, a modelling choice, as in Course 4)
- **BILL_AMT1** to **BILL_AMT6**: the bill, NT dollars, September back to April
- **PAY_AMT1** to **PAY_AMT6**: the amount paid, NT dollars, September back to April
- **default payment next month**: 1 if the account defaulted the following month, **the target**
- **ID**, **SEX**, **EDUCATION**, **MARRIAGE**, **AGE**: never used

READING THE CARD FILE

```
card = pd.read_csv(DATA + "credit_card_default.csv")
TARGET = "default payment next month"
FEATURES = list(X.columns)
```

ROWS AND COLUMNS

```
print(card.shape)
```

```
(30000, 25)
```

The file Course 4 used, unchanged, so we rebuild its rows exactly.

Rebuild Course 4's training, validation and test rows.

Data Preparation for Modeling

COURSE 4'S SPLIT

```
X_rest, X_test, y_rest, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=SEED)
X_train, X_valid, y_train, y_valid = train_test_split(X_rest, y_rest, test_size=0.25, stratify=y_rest, random_state=SEED)
print("Training:", len(X_train), " Validation:", len(X_valid), " Test:", len(X_test))
print("Default share:", round(y_train.mean(), 4), round(y_valid.mean(), 4), round(y_test.mean(), 4))
folds = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
```

```
Training: 18000  Validation: 6000  Test: 6000
Default share: 0.2212 0.2212 0.2212
```

THE THREE KINDS OF ROWS

The same accounts as Course 4 and the days before, each set with the same default share.

Models are reported on the validation rows; Course 5 chooses nothing on them and never scores the test rows.

Training rows fit and choose, validation rows report, test rows stay closed.

Fit LightGBM's defaults and score both row sets.

Model Building

LIGHTGBM IN ONE LINE

LightGBM is Day 7's boosting on the log-odds of default, as Day 8's XGBoost runs it: every account starts at the training default rate, and each tree, fitted to what the trees before it still miss, adds a step.

```
lgbm = LGBMClassifier(random_state=SEED, n_jobs=1, verbose=-1).fit(X_train, y_train)
```

TRAINING AND VALIDATION ROWS

	TRAINING	VALIDATION
ROC AUC	0.8979	0.7726
Accuracy	0.8466	0.8207
Recall	0.4350	0.3572
Precision	0.7718	0.6801
F1	0.5564	0.4684

THE SETTINGS IT CHOSE

```
params = lgbm.get_params()
print({name: params[name] for name in ["n_estimators", "learning_rate", "num_leaves", "max_depth", "min_child_samples", "reg_lambda"]})
```

```
{'n_estimators': 100, 'learning_rate': 0.1, 'num_leaves': 31, 'max_depth': -1, 'min_child_samples': 20, 'reg_lambda': 0.0}
```

Out of the box LightGBM ranks unseen accounts better than XGBoost's defaults and memorises less. Its trees are built another way.

How LightGBM Grows a Tree

A QUESTION TO ANSWER FIRST

Q. The first tree stops at 31 leaves. How many questions down is its deepest leaf? Write down a number.

ANOTHER QUESTION

Q. How many different thresholds do the 100 trees use on **BILL_AMT1**? Write down a number.

EVERY NODE AS A ROW

.booster_ hands back the LightGBM model inside, and its **.trees_to_dataframe()** lists every node of every tree, one row each, as Day 8's XGBoost table did.

THE NODES IN CODE

```
trees = lgbm.booster_.trees_to_dataframe()
first = trees[trees["tree_index"] == 0].set_index("node_index")
```

THE LEAVES IN CODE

```
leaves = first[first["split_feature"].isna()]
print("Leaves:", len(leaves), " Deepest leaf:", leaves["node_depth"].max() - 1, "questions down")
print("Accounts in a leaf: fewest", leaves["count"].min(), " most", leaves["count"].max())
```

Three Settings by Five Folds

THE QUESTION

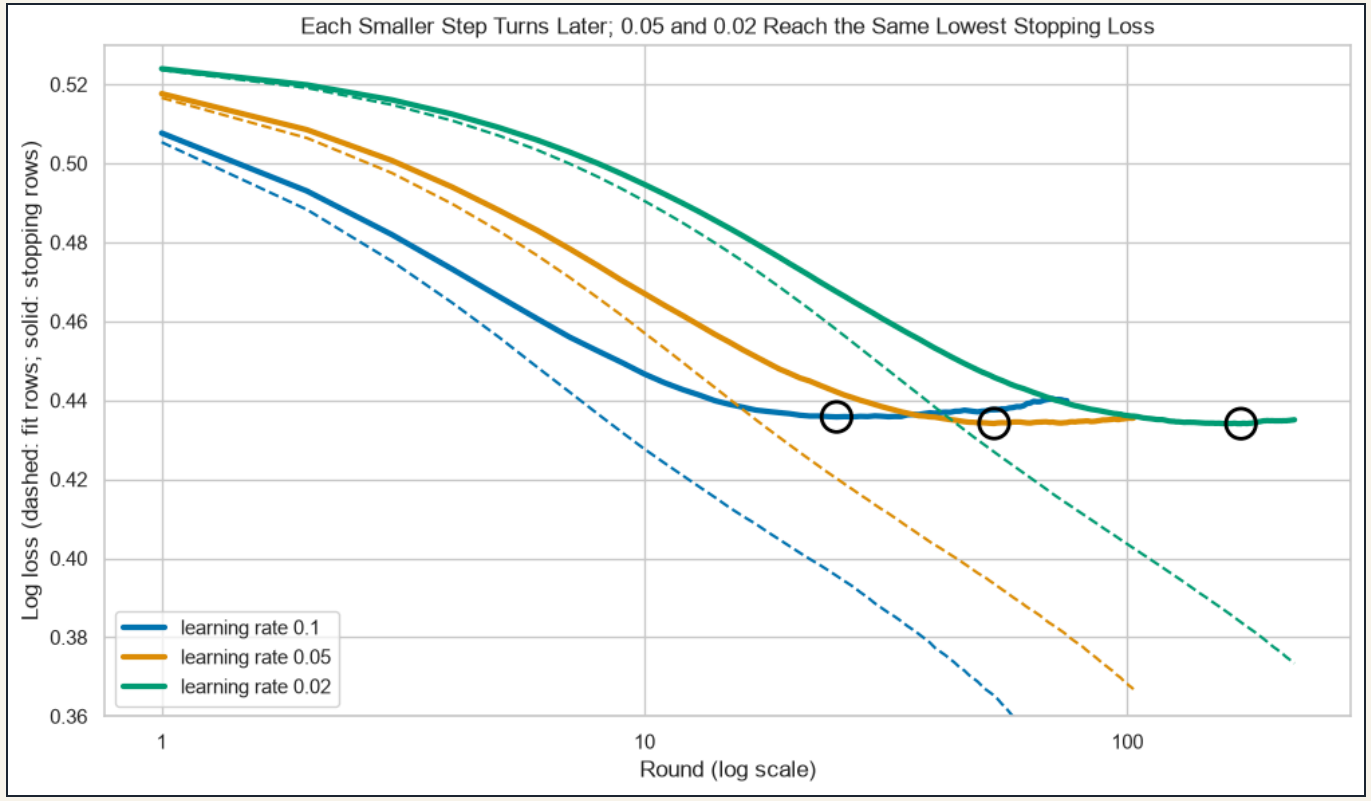
The defaults' 31 leaves score 0.7749 on the held-out folds.

Q. Sketch ROC AUC on the fitted folds and on the held-out folds as **num_leaves** goes 4, 8, 16, 31, 63, 127 and 255.

THREE SETTINGS, FIVE FOLDS EACH

```
settings = {"num_leaves": [4, 8, 16, 31, 63, 127, 255], "min_child_samples": [5, 20, 50, 100, 200, 400, 800],
            "learning_rate": [0.01, 0.03, 0.1, 0.3, 1.0]}
board = []
for setting, values in settings.items():
    for value in values:
        result = cross_validate(LGBMClassifier(random_state=SEED, n_jobs=1, verbose=-1, **{setting: value}), X_train, y_train,
                                cv=folds, scoring="roc_auc", return_train_score=True, n_jobs=1)
        board.append([setting, value, result["train_score"].mean(), result["test_score"].mean(), result["test_score"].std()])
board = pd.DataFrame(board, columns=["setting", "value", "fitted folds", "held-out folds", "fold sd"])
```

Early Stopping on the Stopping Rows

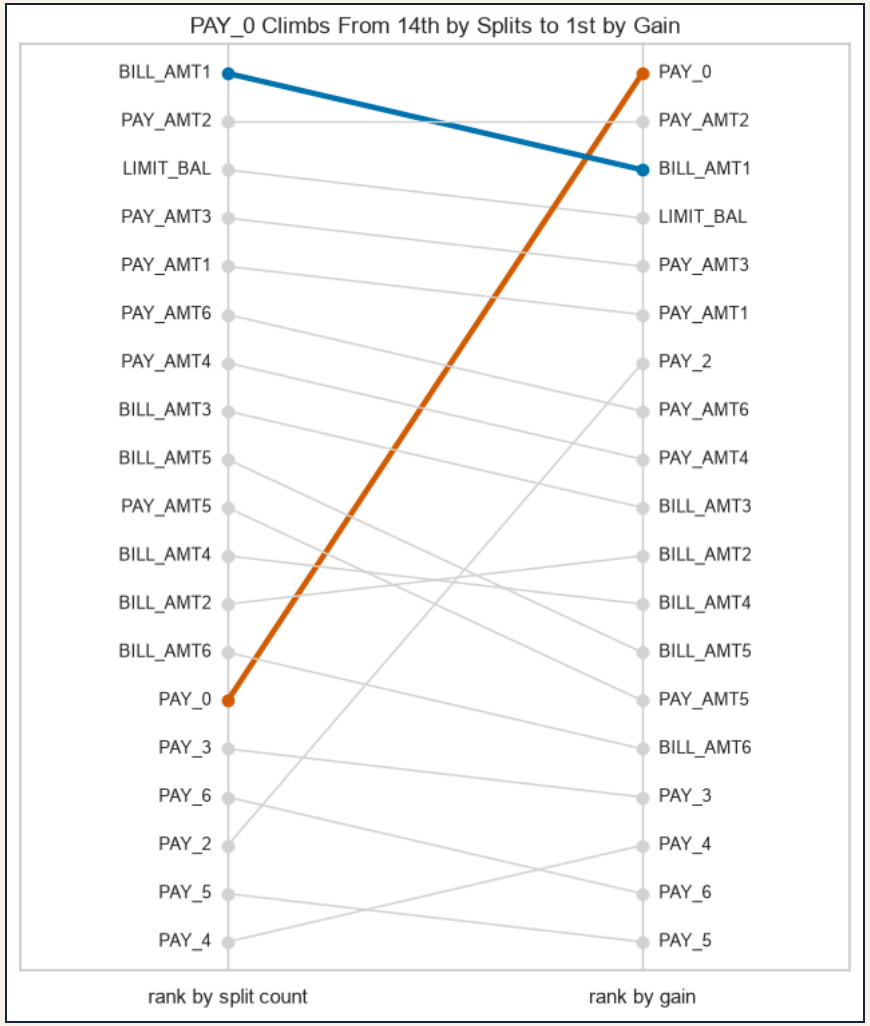


TREES KEPT AT EACH STEP

	TREES KEPT	VALIDATION ROC AUC
0.1000	25	0.7765
0.0500	53	0.7797
0.0200	172	0.7788

At 0.05 the stopping rows keep 53 trees: 53 trees at 0.05 is today's LightGBM from here on.

What LightGBM Leans On



TWO BUILT-IN IMPORTANCES

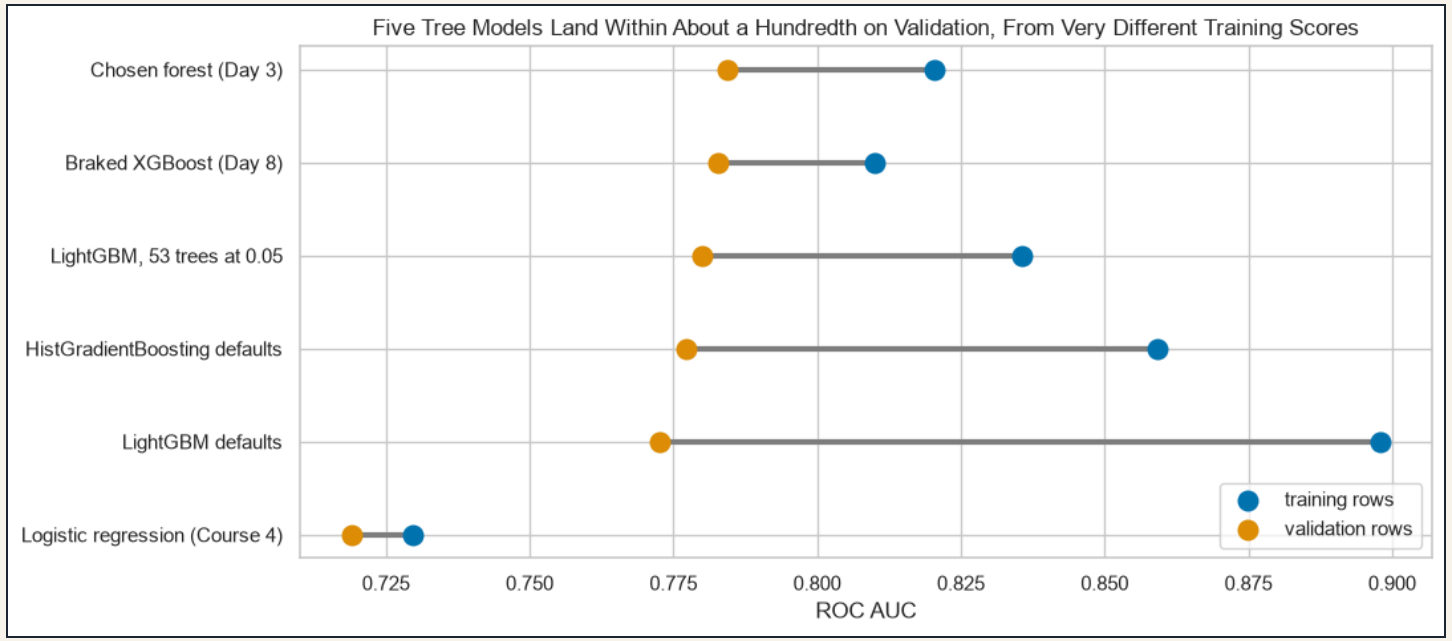
	SPLITS	GAIN PERCENT	SPLIT RANK	GAIN RANK
BILL_AMT1	278	5.7473	1	3
LIMIT_BAL	244	5.6154	3	4
PAY_AMT2	259	6.1598	2	2
PAY_0	114	40.3758	14	1

EACH COLUMN'S SHARE OF THE GAIN

```
importance["gain percent"] = importance["gain"] / importance["gain"].sum() * 100
```

Name `importance_type` whenever you quote LightGBM's importance.

Model Performance Comparison



SCIKIT-LEARN'S OWN BOOSTER

```
Trees: 55  Validation ROC AUC: 0.7773
```

SIX MODELS, VALIDATION ROC AUC

	VALIDATION
Logistic regression (Course 4)	0.7190
LightGBM defaults	0.7726
HistGradientBoosting defaults	0.7773
LightGBM, 53 trees at 0.05	0.7799
Braked XGBoost (Day 8)	0.7827
Chosen forest (Day 3)	0.7844

Five tree models land within 0.0118 on validation, each by a different route. Next we ask whether a model built on top of them beats the best.

Stacking From Out-of-Fold Outputs

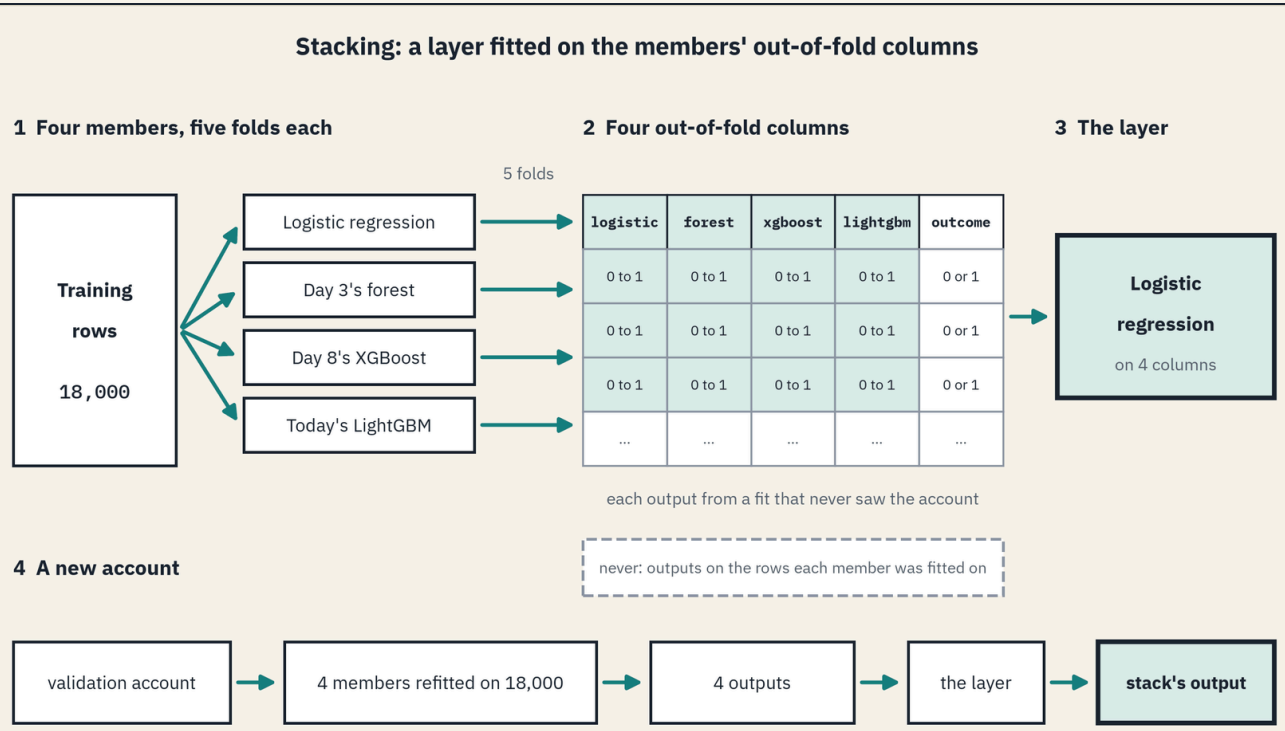


FIGURE 2 Four members' out-of-fold columns feed a logistic layer; members refitted on all 18,000 rows score a new account.

FOUR COLUMNS

	FOREST	XGBOOST	LIGHTGBM
10574	0.1897	0.1947	0.1794
11330	0.1040	0.0912	0.0873
7127	0.0286	0.0461	0.0470

THE LAYER

```
layer = LogisticRegression().fit(oof, y_train)
```

STACKINGCLASSIFIER AGAINST THE HAND LAYER

```
Largest gap between the two layers' coefficients: 0.0
```

The stack is the four columns, a logistic layer and four refits, in one object.

Does the Stack Beat Its Best Member?

THE QUESTION

Q. The best member, Day 3's chosen forest, scores 0.7844 on the validation rows. What will the stack score? Write down a number.

SCORING THE STACK

```
stack_out = stack.predict_proba(X_valid)[:, 1]
```

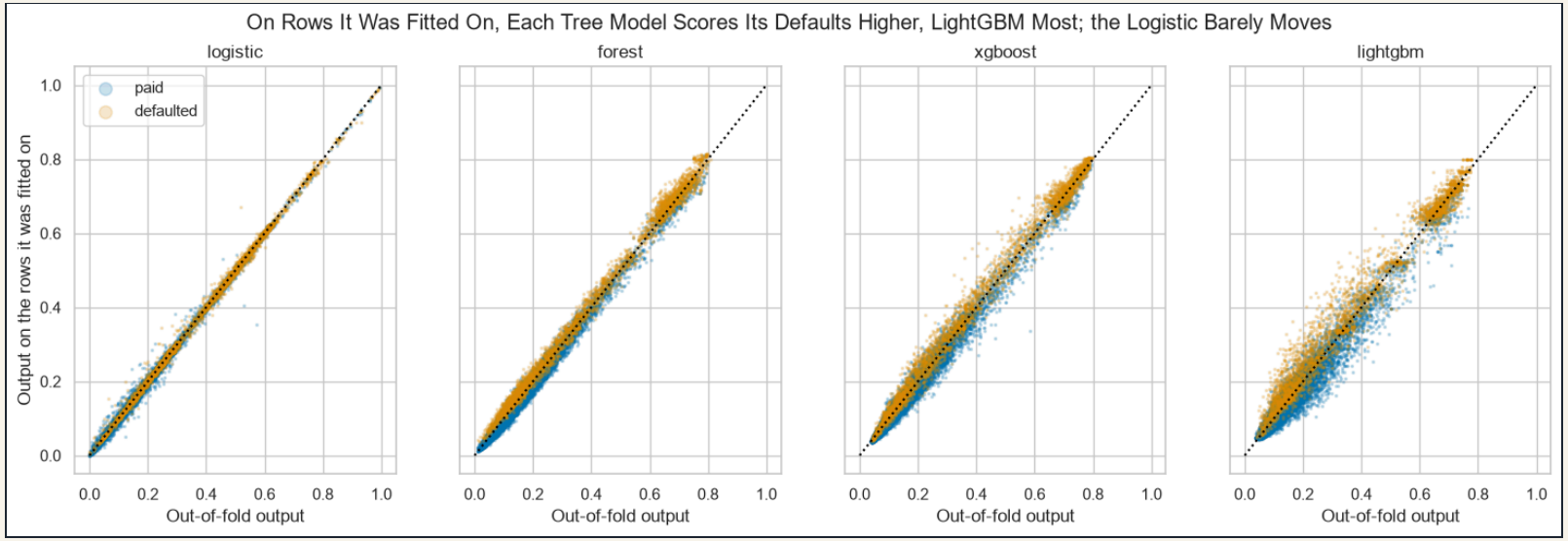
EVERY BLEND OF THREE

```
trio = oof[["forest", "xgboost", "lightgbm"]]
blends = []
for a in range(21):
    for b in range(21 - a):
        weights = np.array([a, b, 20 - a - b]) / 20
        blends.append(list(weights) + [roc_auc_score(y_train, (trio * weights).sum(axis=1))])
blends = pd.DataFrame(blends, columns=["forest", "xgboost", "lightgbm", "ROC AUC"])
```

A blend weights the members' outputs by numbers we choose; the plain average gives each the same weight.

Sum up the day in six findings.

Insights and Conclusion



THE MEMBERS ON THEIR OWN FITTED ROWS

```
in_sample = pd.DataFrame({name: model.predict_proba(X_train)[:, 1] for name, model in models.items()}, index=X_train.index)
print({name: round(roc_auc_score(y_train, in_sample[name]), 4) for name in in_sample})
```

```
{'logistic': 0.7296, 'forest': 0.8204, 'xgboost': 0.81, 'lightgbm': 0.8355}
```

Day 10 puts an interval on every gap before naming a winner.

You Can Now

- Fit **`LGBMClassifier`** on one thread and read its trees with **`.trees_to_dataframe()`**.
- Draw a tree as an icicle chart, and count the thresholds binned columns allow.
- Score settings on five folds and stop LightGBM's trees on stopping rows with a callback.
- Read LightGBM's split-count and gain importances and name which one you quote.
- Build out-of-fold columns, fit a stacking layer on them, and check **`StackingClassifier`** against it.

NEXT
WEEK 2 · DAY 5

Case Study: Boosting on Credit Card Default

Day 10 puts an interval on every gap before naming a winner.